

**LAPORAN PRAKTIKUM PEMROGRAMAN KOMPUTER LANJUT
PRESENSI MAHASISWA BERBASIS DATABASE DARI PHPMYADMIN**



Disusun oleh :

Muhammad Athaozih (23090620043)

Ahmad Ilman Nadziron (23090620065)

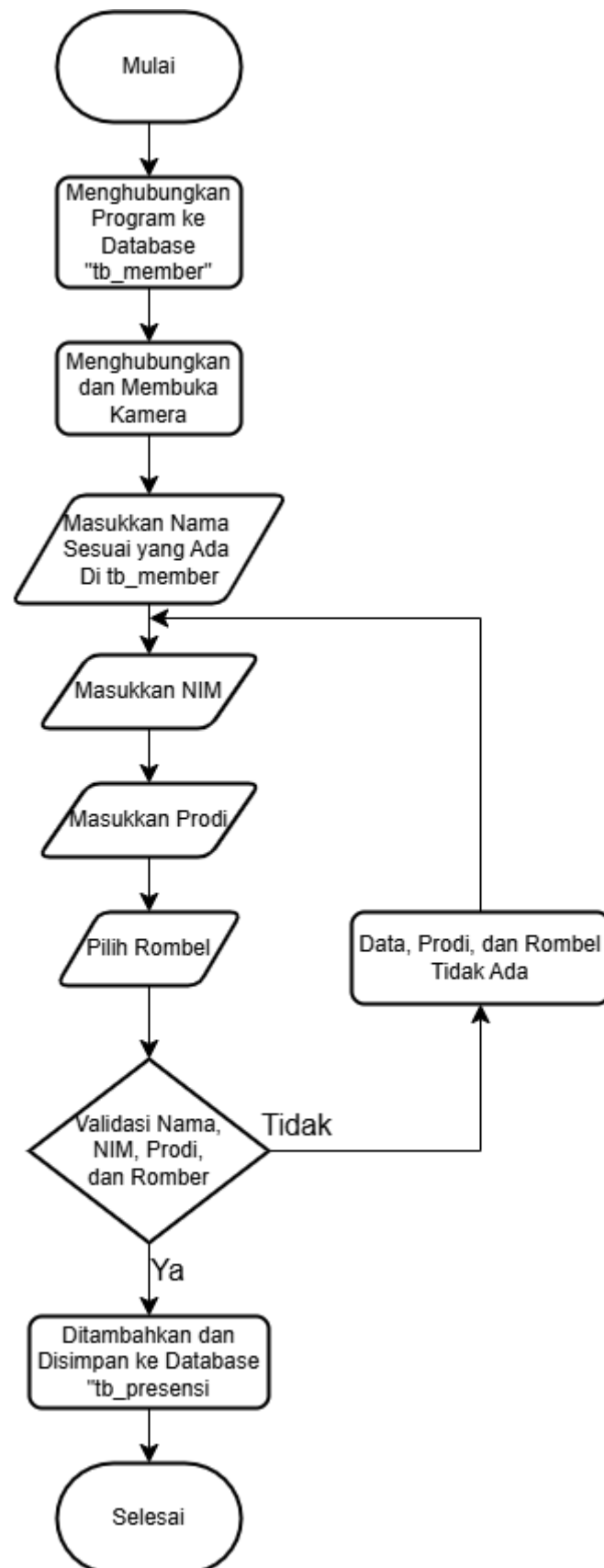
Zacky Allamsyah Bintoro (23090620078)

Anjani Nuria Arsyla (23090620084)

**D4 TEKNIK ELEKTRONIKA
DEPARTEMEN ELEKTRO DAN ELEKTRONIKA
FAKULTAS VOKASI
UNIVERSITAS NEGERI YOGYAKARTA**

2024

A. Flowchart



Gambar 1. Flowchart

B. Kode Program

```
from email import message
import tkinter
import cv2
from PIL import Image, ImageTk
from datetime import datetime
import os
import pygame
import mysql.connector
from tkinter import ttk

# Initialize pygame mixer for sound
pygame.mixer.init()

# Create application window
tk = tkinter.Tk()
tk.geometry('1000x900') # Increased window height to
accommodate larger image
tk.title("Presensi Universitas Negeri Yogyakarta")
tk.configure(bg="white")

def update_time():
    now = datetime.now()
    formatted_time = now.strftime("%A, %d %B %Y |
%H:%M:%S")
    label_time.config(text=formatted_time)
    tk.after(1000, update_time)

# Create header frame
header_frame = tkinter.Frame(tk, bg="#1e3a8a",
width=1500, height=140)
header_frame.pack(fill="x")

# Add time label (right-aligned)
label_time = tkinter.Label(header_frame, font=("Arial",
14), fg="white", bg="#1e3a8a")
label_time.pack(side="right", padx=20, pady=10)
update_time()

# Left-aligned logo container
logo_container = tkinter.Frame(header_frame,
bg="#1e3a8a")
logo_container.pack(side="left", padx=20)

# Try to load and display larger logo
try:
    logo_img = Image.open("Logo_UNY.png")
    logo_img = logo_img.resize((100, 100))
    logo_photo = ImageTk.PhotoImage(logo_img)
    logo_label = tkinter.Label(logo_container,
```

```

image=logo_photo, bg="#1e3a8a")
    logo_label.image = logo_photo
    logo_label.pack()
except:
    logo_label = tkinter.Label(logo_container,
text="LOGO", font=("Arial", 14, "bold"),
                                width=10, height=5,
bg="white", fg="#1e3a8a")
    logo_label.pack()

# Center header text container
header_text = tkinter.Frame(header_frame, bg="#1e3a8a")
header_text.pack(expand=True, pady=10)

# Header labels
welcome_label = tkinter.Label(header_text, text="Selamat
Datang di Laman",
                                font=("Arial", 18),
fg="white", bg="#1e3a8a")
welcome_label.pack()

university_label = tkinter.Label(header_text,
text="Universitas Negeri Yogyakarta",
                                font=("Arial", 24,
"bold"), fg="#ffd700", bg="#1e3a8a")
university_label.pack()

faculty_label = tkinter.Label(header_text,
text="Fakultas Vokasi",
                                font=("Arial", 22, "bold"),
fg="white", bg="#1e3a8a")
faculty_label.pack()

department_label = tkinter.Label(header_text,
text="Departemen Elektro dan Elektronika",
                                font=("Arial", 20),
fg="white", bg="#1e3a8a")
department_label.pack()

# Initialize camera with higher resolution
vid = cv2.VideoCapture(0)
vid.set(cv2.CAP_PROP_FRAME_WIDTH, 1100)
vid.set(cv2.CAP_PROP_FRAME_HEIGHT, 550)

def create_placeholder_image(width, height, text="Foto
akan muncul di sini"):
    # Create a blank image with specified size
    img = Image.new('RGB', (width, height),
color='lightgray')
    from PIL import ImageDraw

```

```

draw = ImageDraw.Draw(img)

# Add text to the center of image
try:
    from PIL import ImageFont
    font = ImageFont.truetype("arial.ttf", 14)
except:
    font = ImageFont.load_default()

text_bbox = draw.textbbox((0, 0), text, font=font)
text_width = text_bbox[2] - text_bbox[0]
text_height = text_bbox[3] - text_bbox[1]

x = (width - text_width) / 2
y = (height - text_height) / 2

# Draw text in dark gray
draw.text((x, y), text, fill='#666666', font=font)

return img

def validate_prodi(prodi):
    config = {
        'user': 'root',
        'password': '',
        'host': '127.0.0.1',
        'database': 'db_presensi',
        'raise_on_warnings': True
    }
    try:
        cnx = mysql.connector.connect(**config)
        if cnx.is_connected():
            cursor = cnx.cursor()
            query = "SELECT * FROM tb_member WHERE prodi
= %s"
            cursor.execute(query, (prodi,))
            result = cursor.fetchall()
            cnx.close()
            return len(result) > 0 # Jika prodi
ditemukan, return True
        except mysql.connector.Error as err:
            print(f"Error: {err}")
            return False

# Fitur tambahan: Validasi Rombel (V1, V2, V3)
def validate_rombel(rombel):
    config = {
        'user': 'root',
        'password': '',
        'host': '127.0.0.1',

```

```

        'database': 'db_presensi',
        'raise_on_warnings': True
    }

    cnx = mysql.connector.connect(**config)
    if cnx.is_connected():
        cursor = cnx.cursor()
        query = "SELECT * FROM tb_member WHERE rombel =
%s"
        cursor.execute(query, (rombel,))
        result = cursor.fetchall()
        cnx.close()
        return len(result) > 0 # Jika rombel ada,
return True
    return False

def play_sound(success=True):
    if success:
        # Play a success sound (you can use your own
sound file)
        pygame.mixer.Sound("Sound_benar.wav").play()
    else:
        # Play an error sound (you can use your own
sound file)
        pygame.mixer.Sound("Sound_salah.wav").play()

def process():
    global img
    global image_
    nim = entry.get() # Ambil input NIM
    nama = entry1.get() # Ambil input Nama (label Nama)
    prodi = entry2.get()
    rombel = rombel_combobox.get() # Ambil input rombel

    if not validate_rombel(rombel):
        notif1.set('Rombel tidak valid!')
        play_sound(success=False)
        return

    # Validasi Prodi
    if not validate_prodi(prodi):
        notif1.set('Prodi tidak valid!')
        play_sound(success=False)
        return

    if nim.isnumeric(): # Pastikan NIM hanya berisi
angka
        config = {
            'user': 'root',
            'password': '',

```

```

        'host': '127.0.0.1',
        'database': 'db_presensi',
        'raise_on_warnings': True
    }

    cnx = mysql.connector.connect(**config)
    if cnx.is_connected():
        cursor = cnx.cursor()

        # Query untuk mengecek apakah NIM dan Nama
ada di database
        query = "SELECT * FROM tb_member WHERE
id_nim = %s AND nama_mahasiswa = %s"
        cursor.execute(query, (nim, nama))
        rows = cursor.fetchall()

        if len(rows) == 0: # Jika NIM atau Nama
tidak ditemukan
            notif1.set('Data tidak sesuai atau tidak
ada')
            play_sound(success=False)
        else:
            # Jika data valid, lanjutkan proses
presensi
            _, image = vid.read()
            current_datetime = datetime.now()
            formatted_datetime =
current_datetime.strftime("%Y%m%d%H%M%S")
            current_time =
current_datetime.strftime("%H:%M:%S")
            current_date =
current_datetime.strftime("%Y-%m-%d")
            current_day =
current_datetime.strftime("%A")

            # Simpan foto hasil presensi
            x1, y1 = 425, 200
            x2, y2 = 925, 700
            cropped_image = image[y1:y2, x1:x2]

            cv2.imwrite(f"file_{formatted_datetime}.png",
cropped_image)

            name_file =
f"file_{formatted_datetime}.png"
            sql = "INSERT INTO tb_presensi (id_user,
hari_kedatangan, tanggal_kedatangan, waktu_kedatangan,
image) VALUES (%s, %s, %s, %s, %s)"
            val = (nim, current_day, current_date,
current_time, name_file)

```

```

        cursor.execute(sql, val)
        cnx.commit()

        # Tampilkan data presensi
        for row in rows:
            notif1.set('Nama: ' + row[1]) #
Menampilkan nama
            notif2.set('NIM: ' + row[0]) #
Menampilkan NIM
            notif3.set('Waktu: ' + current_time)
            notif4.set('Hari: ' + current_day)
            notif5.set('Tanggal: ' +
current_date)

            img =
Image.open(f"file_{formatted_datetime}.png")
            resized_image = img.resize((125, 160))
            image_ =
ImageTk.PhotoImage(resized_image)
            canvas1.create_image(40, 80,
image=image_)
            notif6.set('Anda Berhasil Presensi')
            play_sound(success=True)

        cnx.close()
    else:
        notif1.set('Koneksi ke database gagal')
        play_sound(success=False)
    else:
        notif1.set('Masukkan NIM dengan benar')
        play_sound(success=False)

# Fungsi untuk mengisi daftar nama dari database ke
Label Nama
def fill_nama_from_database():
    config = {
        'user': 'root',
        'password': '',
        'host': '127.0.0.1',
        'database': 'db_presensi',
        'raise_on_warnings': True
    }

    try:
        cnx = mysql.connector.connect(**config)
        if cnx.is_connected():
            cursor = cnx.cursor()
            # Query untuk mengambil semua nama di
database

```

```

        query = "SELECT nama FROM tb_member"
        cursor.execute(query)
        rows = cursor.fetchall()

        # Isi nama-nama ke drop-down atau pilihan
input
        nama_list = [row[0] for row in rows]
        nama_menu['values'] = nama_list # Mengisi
ComboBox dengan nama

        cnx.close()
    except mysql.connector.Error as err:
        print(f"Error: {err}")

def open_camera():
    """Update camera feed."""
    ret, frame = vid.read()
    opencv_image = cv2.cvtColor(frame,
cv2.COLOR_BGR2RGBA)
    captured_image = Image.fromarray(opencv_image)
    photo_image =
ImageTk.PhotoImage(image=captured_image)
    canvas.create_image(100, 0, image=photo_image,
anchor=tkinter.CENTER)
    canvas.image = photo_image
    tk.after(10, open_camera)

notif1 = tkinter.StringVar()
notif2 = tkinter.StringVar()
notif3 = tkinter.StringVar()
notif4 = tkinter.StringVar()
notif5 = tkinter.StringVar()
notif6 = tkinter.StringVar()

notif1.set('')
notif2.set('')
notif3.set('')
notif4.set('')
notif5.set('')
notif6.set('')

canvas = tkinter.Canvas(tk, width=360, height=350)
canvas.pack(anchor=tkinter.W)
canvas.place(x=700, y=300)

canvas1 = tkinter.Canvas(tk, width=100, height=120)
canvas1.pack(expand=tkinter.YES)
canvas1.place(x=1200, y=300)

```

```
button = tkinter.Button(tk, text="Presensi", width=30,
height=2, command=process)
button.place(x=1200, y=740)

entry = tkinter.Entry(tk, width=29)
entry.place(x=1200,y=550)

entry1 = tkinter.Entry(tk, width=29)
entry1.place(x=1200,y=490)

entry2 = tkinter.Entry(tk, width=29)
entry2.place(x=1200,y=610)

label_input = tkinter.Label(tk, text="Masukkan NIM")
label_input.place(x=1270,y=520)

label_input1 = tkinter.Label(tk, text="Masukkan Nama")
label_input1.place(x=1270,y=460)

# Add ComboBox untuk Rombel
label_input2 = tkinter.Label(tk, text="Pilih Rombel")
label_input2.place(x=1270, y=640)
rombel_combobox = ttk.Combobox(tk, values=["V1", "V2",
"V3"], width=27)
rombel_combobox.place(x=1200, y=670)

# Tambahkan ComboBox untuk Prodi di GUI
label_input_prodi = tkinter.Label(tk, text="Pilih
Prodi")
label_input_prodi.place(x=1270, y=580)

label_notif1 = tkinter.Label(tk, textvariable=notif1)
label_notif1.place(x=1330,y=300)

label_notif2 = tkinter.Label(tk, textvariable=notif2)
label_notif2.place(x=1330,y=330)

label_notif3 = tkinter.Label(tk, textvariable=notif3)
label_notif3.place(x=1330,y=360)

label_notif4 = tkinter.Label(tk, textvariable=notif4)
label_notif4.place(x=1330,y=390)

label_notif5 = tkinter.Label(tk, textvariable=notif5)
label_notif5.place(x=1330,y=420)

label_notif6 = tkinter.Label(tk, textvariable=notif6)
label_notif6.place(x=820,y=670)
```

```
# Start camera feed
open_camera()

tk.mainloop()
```

Penjelasan Program

1. from email import message

- Impor modul message dari library email. Modul ini kemungkinan digunakan untuk membaca dan memproses pesan email.

2. import tkinter

- Impor library tkinter yang merupakan modul standar Python untuk membangun Graphical User Interface (GUI).

3. import cv2

- Impor library OpenCV (cv2) yang digunakan untuk pemrosesan gambar dan video. Diperlukan jika program ini nantinya akan melibatkan fungsi kamera atau pemrosesan gambar.

4. from PIL import Image, ImageTk

- Impor modul Image dan ImageTk dari library Pillow (PIL Fork).
- Image digunakan untuk pemrosesan gambar.
- ImageTk digunakan untuk konversi gambar dari format PIL ke format yang dapat ditampilkan di widget Tkinter.

5. from datetime import datetime

- Impor modul datetime untuk mendapatkan informasi tanggal dan waktu.

6. import os

- Impor modul os untuk melakukan operasi sistem, seperti mengakses file dan direktori.

7. import pygame

- Impor library pygame untuk menangani audio dan multimedia. Diperlukan jika program ini nantinya akan menggunakan suara atau efek suara.

8. import mysql.connector

- Impor modul mysql.connector untuk koneksi ke database MySQL.

9. from tkinter import ttk

- Impor modul ttk dari library Tkinter yang menyediakan widget tambahan (seperti ttk.Combobox) untuk mempercantik antarmuka.

10. pygame.mixer.init()

- Inisialisasi modul `pygame.mixer` untuk menangani pemutaran suara.
11. `tk = tkinter.Tk()`
 - Membuat instance dari `tkinter.Tk()` untuk menjadi window utama dari aplikasi.
 12. `tk.geometry('1000x900')`
 - Mengatur ukuran window utama menjadi 1000 pixel x 900 pixel (tinggi jendela dinaikkan untuk mengakomodasi gambar yang lebih besar).
 13. `tk.title`
 - Digunakan untuk presensi ("Presensi Universitas Negeri Yogyakarta")
 14. `tk.configure(bg="white")`
 - Mengubah warna background window utama menjadi putih.
 15. `def update_time():`
 - Mendefinisikan sebuah fungsi bernama `update_time()`.
 16. `now = datetime.now()`
 - Mendapatkan waktu saat ini menggunakan fungsi `now()` dari modul `datetime` dan menyimpannya dalam variabel `now`.
 17. `formatted_time = now.strftime("%A, %d %B %Y | %H:%M:%S")`
 - Memformat waktu yang didapat dari `now` menjadi format yang mudah dibaca manusia menggunakan fungsi `strftime()`. Format yang digunakan sama persis dengan yang ditampilkan sebelumnya (baris 3). Hasil format disimpan dalam variabel `formatted_time`.
 18. `label_time.config(text=formatted_time)`
 - Mengubah teks pada label bernama `label_time` (yang belum didefinisikan sebelumnya). Teks yang ditampilkan adalah `formatted_time` yang berisi waktu yang sudah diformat.
 - `label_time.config` digunakan untuk mengkonfigurasi properti dari label `label_time`.
 - `text` adalah properti yang diubah, dan nilainya diganti dengan `formatted_time`.
 19. `tk.after(1000, update_time)`
 - Menggunakan fungsi `after` dari modul `tkinter` untuk menjadwalkan pemanggilan fungsi `update_time()` kembali setelah 1000 milidetik (1 detik).
 - memastikan fungsi `update_time` dipanggil berulang kali untuk memperbarui waktu secara real-time pada label.
 20. `# Create header frame`

- menandakan blok kode selanjutnya akan membuat frame untuk header aplikasi.

21. `header_frame = tkinter.Frame(tk, bg="#1e3a8a", width=1500, height=140)`

- Membuat sebuah frame (container) bernama `header_frame` menggunakan `tkinter.Frame`.
- `tk` adalah referensi ke window utama yang dibuat sebelumnya.

22. `header_frame.pack(fill="x")`

- Menambahkan frame `header_frame` ke dalam window utama (`tk`).
- `pack` adalah method yang digunakan untuk mengatur layout elemen (widget) dalam window.
- `fill="x"` menginstruksikan agar frame tersebut mengisi seluruh lebar window secara horizontal.

23. `# Add time label (right-aligned)`

- Komentar yang menandakan blok kode selanjutnya akan membuat label untuk menampilkan waktu, diposisikan rata kanan.

24. `label_time = tkinter.Label(header_frame, font=("Arial", 14), fg="white", bg="#1e3a8a")`

- Membuat sebuah label bernama `label_time` menggunakan `tkinter.Label`.
- `header_frame` adalah referensi ke frame yang dibuat sebelumnya, digunakan sebagai parent container untuk label.
- `font=("Arial", 14)` menを設定 (shè zhì) (seting) font yang digunakan untuk menampilkan teks pada label menjadi Arial dengan ukuran 14.
- `fg="white"` menを設定 (shè zhì) (seting) warna teks pada label menjadi putih.
- `bg="#1e3a8a"` menを設定 (shè zhì) (seting) warna background label (sama dengan warna frame header).

25. `label_time.pack(side="right", padx=20, pady=10)`

- Menambahkan label `label_time` ke dalam frame `header_frame`.
- `pack` adalah method yang digunakan untuk mengatur layout elemen (widget) dalam frame.
- `side="right"` menginstruksikan agar label ditempatkan di sisi kanan frame.
- `padx=20, pady=10` menambahkan padding (jarak) sebesar 20 pixel ke kanan dan kiri teks (`padx`), serta 10 pixel ke atas dan bawah teks (`pady`).

26. `update_time()`

- Setelah mengatur tampilan label waktu, fungsi `update_time()` dipanggil langsung untuk memulai proses pembaruan waktu secara otomatis.

27. `# Left-aligned logo container`

- komentar yang menandai awal dari pembuatan sebuah container untuk menempatkan logo.

28. `logo_container = tkinter.Frame(header_frame, bg="#1e3a8a")`

- Membuat sebuah frame baru bernama `logo_container` yang akan menjadi wadah untuk logo. Frame ini diletakkan di dalam `header_frame`. Warna latar belakangnya sama dengan `header_frame`.

29. `logo_container.pack(side="left", padx=20)`

- Menambahkan frame `logo_container` ke dalam `header_frame`.
- `side="left"`: Menempatkan `logo_container` di sisi kiri dalam `header_frame`.
- `padx=20`: Memberikan jarak 20 pixel di sebelah kiri dan kanan `logo_container`.

30. `# Try to load and display larger logo`

- Komentar ini menandakan blok kode selanjutnya bertujuan untuk memuat dan menampilkan logo berukuran lebih besar.

31-35. try block

- Blok try digunakan untuk mencoba menjalankan kode di dalamnya. Jika terjadi kesalahan (exception), maka blok except akan dijalankan sebagai gantinya.
- `logo_img = Image.open("Logo_UNY.png")`: Membuka gambar bernama "Logo_UNY.png" menggunakan modul Image dari Pillow (PIL Fork).
- `logo_img = logo_img.resize((100, 100))`: Mengubah ukuran gambar menjadi 100 pixel x 100 pixel menggunakan method `resize`.
- `logo_photo = ImageTk.PhotoImage(logo_img)`: Mengubah gambar dari format PIL menjadi format yang dapat ditampilkan di widget Tkinter menggunakan `ImageTk.PhotoImage`.
- `logo_label = tkinter.Label(logo_container, image=logo_photo, bg="#1e3a8a")`: Membuat sebuah label bernama `logo_label` untuk menampilkan gambar logo.
- `logo_container` adalah referensi ke container yang sudah dibuat sebelumnya.
- `image=logo_photo`: Menentukan gambar yang akan ditampilkan pada label menggunakan referensi `logo_photo`.
- `bg="#1e3a8a"`: Menentukan (shè zhì) (setting) warna background label menjadi sama dengan container.
- `logo_label.image = logo_photo`: Ini sebenarnya tidak diperlukan, karena properti `image` sudah diatur pada baris sebelumnya.
- `logo_label.pack()`: Menambahkan label `logo_label` ke dalam container `logo_container`.

36. except:

- Blok except akan dijalankan jika terjadi kesalahan saat membuka atau memproses gambar logo.

37-42. except block

- Blok ini akan menampilkan label dengan teks "LOGO" jika gambar logo gagal dimuat.
- `logo_label = tkinter.Label(logo_container, text="LOGO", ...)`: Membuat label `logo_label` dengan teks "LOGO" sebagai pengganti jika gambar tidak ditemukan.
- Properti lainnya seperti font, ukuran, dan background color diatur sesuai keinginan.
- `logo_label.pack()`: Menambahkan label `logo_label` ke dalam container `logo_container`.

43. # Center header text container

- Komentar ini menandakan blok kode selanjutnya akan membuat container untuk teks header yang dipusatkan secara horizontal.

44. `header_text = tkinter.Frame(header_frame, bg="#1e3a8a")`

- Membuat sebuah frame bernama `header_text` yang akan menampung label-label header aplikasi. Frame ini berada di dalam `header_frame`.

45. `header_text.pack(expand=True, pady=10)`

- Menambahkan frame `header_text` ke dalam `header_frame`.
- `expand=True`: Menginstruksikan agar frame ini mengisi seluruh sisa space yang ada secara horizontal di dalam `header_frame`.
- `pady=10`: Memberikan padding (jarak) 10 pixel ke atas dan bawah elemen-elemen yang ditempatkan di dalam `header_text`.

46-49. Header Labels

- Blok kode selanjutnya membuat dan menambahkan beberapa label ke container `header_text`. Label-label tersebut menampilkan informasi header aplikasi seperti "Selamat Datang di Laman", nama Universitas, Fakultas, dan Departemen.
- Setiap label dibuat menggunakan `tkinter.Label` dengan properti seperti teks, font, dan warna yang disesuaikan.
- Method `pack` digunakan untuk menambahkan setiap label ke dalam container `header_text`.

50. # Initialize camera with higher resolution

- menandakan blok kode selanjutnya bertujuan untuk menginisialisasi kamera dengan resolusi yang lebih tinggi.

51. `vid = cv2.VideoCapture(0)`

- Membuka kamera default (biasanya webcam) dan menyimpan referensi ke kamera tersebut dalam variabel `vid` menggunakan `cv2.VideoCapture(0)`.

- 52-53. `vid.set(cv2.CAP_PROP_FRAME_WIDTH, 1100) & vid.set(cv2.CAP_PROP_FRAME_HEIGHT, 550)`
 - Mengatur resolusi kamera yang dibuka sebelumnya.
 - `vid.set`: Digunakan untuk mengatur properti kamera.
 - `cv2.CAP_PROP_FRAME_WIDTH`: Konstanta untuk properti lebar frame (dimensi horizontal).
 - 1100: Menyetel lebar frame kamera menjadi 1100 pixel.
 - `vid.set`: Digunakan kembali untuk mengatur properti kamera.
 - `cv2.CAP_PROP_FRAME_HEIGHT`: Konstanta untuk properti tinggi frame (dimensi vertikal).
 - 550: Menyetel tinggi frame kamera menjadi 550 pixel.
54. `def create_placeholder_image(width, height, text="Foto akan muncul di sini"):`
- Mendefinisikan fungsi bernama `create_placeholder_image` yang digunakan untuk membuat gambar placeholder (pengganti) dengan teks.
- 55-56. `img = Image.new('RGB', (width, height), color='lightgray')`
- Membuat sebuah image kosong dengan ukuran width x height pixel dan warna latar belakang 'lightgray' menggunakan `Image.new`.
- 57-60. Menambahkan teks ke gambar placeholder
- Blok kode ini menambahkan teks ke tengah gambar placeholder menggunakan modul Pillow.
61. `return img`
- Mengembalikan image yang telah dibuat dengan teks di dalamnya.
62. `def validate_prodi(prodi):`
- Mendefinisikan fungsi bernama `validate_prodi` yang digunakan untuk memvalidasi program studi (`prodi`) yang diinputkan.
- 63-68. Mendefinisikan konfigurasi untuk koneksi database
- Blok kode ini mendefinisikan detail konfigurasi untuk koneksi ke database MySQL.
69. `try:`
- Blok `try` digunakan untuk mencoba menjalankan kode di dalamnya. Jika terjadi kesalahan (`exception`), maka blok `except` akan dijalankan sebagai gantinya.
70. `cnx = mysql.connector.connect(**config)`
- Mencoba membuat koneksi ke database MySQL menggunakan informasi konfigurasi yang telah didefinisikan sebelumnya.

71-73. Memeriksa koneksi dan menyiapkan kueri (belum selesai)

- Blok kode ini akan mengecek apakah koneksi ke database berhasil dan menyiapkan kueri SQL untuk memvalidasi program studi.

74-78. Memproses hasil kueri dan menutup koneksi

- `cursor.execute(query, (prodi,))`: Mengeksekusi kueri SQL dengan parameter `prodi`.
- `result = cursor.fetchall()`: Mengambil semua hasil kueri dan menyimpannya dalam `result`.
- `cnx.close()`: Menutup koneksi ke database.
- `return len(result) > 0`: Mengembalikan `True` jika ada hasil kueri (`prodi` ditemukan), `False` jika tidak.

79-80. Menangani kesalahan koneksi database

- `except mysql.connector.Error as err`: Blok `except` ini akan dijalankan jika terjadi kesalahan saat koneksi atau eksekusi kueri.
- `print(f"Error: {err}")`: Mencetak pesan kesalahan ke konsol.
- `return False`: Mengembalikan `False` untuk mengindikasikan kegagalan validasi.

81. `def validate_rombel(rombel):`

- Mendefinisikan fungsi `validate_rombel` untuk memvalidasi `rombel`.

82-87. Mendefinisikan konfigurasi database

- Blok kode ini mendefinisikan detail konfigurasi untuk koneksi ke database MySQL.

88. `cnx = mysql.connector.connect(**config)`

- Mencoba membuat koneksi ke database MySQL menggunakan informasi konfigurasi yang telah didefinisikan sebelumnya.

89-93. Memeriksa koneksi dan mengeksekusi kueri

- Blok kode ini mengecek koneksi ke database dan mengeksekusi kueri SQL untuk memvalidasi `rombel`.

94-95. Memproses hasil kueri dan menutup koneksi

- Blok kode ini memproses hasil kueri dan menutup koneksi ke database.

96. `def play_sound(success=True):`

- Mendefinisikan fungsi `play_sound` untuk memutar suara berdasarkan hasil validasi.

97-100. Memutar suara berdasarkan hasil validasi

- `if success::` Jika `success` adalah `True`, memutar suara keberhasilan.

- `pygame.mixer.Sound("Sound_benar.wav").play():` Memutar file suara "Sound_benar.wav".
- `else::` Jika success adalah False, memutar suara kesalahan.
- `pygame.mixer.Sound("Sound_salah.wav").play():` Memutar file suara "Sound_salah.wav".

101. `def process():`

- Mendefinisikan fungsi process untuk memproses input dari pengguna.

102-103. Mengambil input dari pengguna

- `nim = entry.get():` Mengambil nilai NIM dari widget entry.
- `nama = entry1.get():` Mengambil nilai nama dari widget entry1.
- `prodi = entry2.get():` Mengambil nilai prodi dari widget entry2.
- `rombel = rombel_combobox.get():` Mengambil nilai rombel dari combobox rombel_combobox.

104-107. Memvalidasi rombel

- `if not validate_rombel(rombel)::` Jika rombel tidak valid, maka:
- `notif1.set('Rombel tidak valid!')`: Mengatur teks notifikasi menjadi "Rombel tidak valid!".
- `play_sound(success=False):` Memutar suara kesalahan.
- `return:` Menghentikan eksekusi fungsi process.

108-111. Memvalidasi prodi

- `if not validate_prodi(prodi)::` Jika prodi tidak valid, maka:
- `notif1.set('Prodi tidak valid!')`: Mengatur teks notifikasi menjadi "Prodi tidak valid!".
- `play_sound(success=False):` Memutar suara kesalahan.
- `return:` Menghentikan eksekusi fungsi process.

112. `if nim.isnumeric():`

- Memeriksa apakah nilai nim hanya terdiri dari angka. Jika benar, maka akan melanjutkan proses validasi dan penyimpanan data.

113-119. Mendefinisikan konfigurasi database

- Blok kode ini mendefinisikan detail konfigurasi untuk koneksi ke database MySQL.

120. `cnx = mysql.connector.connect(**config)`

- Mencoba membuat koneksi ke database MySQL menggunakan informasi konfigurasi yang telah didefinisikan sebelumnya.

121-125. Memeriksa koneksi dan mengeksekusi kueri

- `if cnx.is_connected():` Memeriksa apakah koneksi ke database berhasil.

- `cursor = cnx.cursor()`: Membuat objek cursor untuk mengeksekusi kueri SQL.
- `query = "SELECT * FROM tb_member WHERE id_nim = %s AND nama_mahasiswa = %s"`: Mendefinisikan kueri SQL untuk mencari data mahasiswa berdasarkan NIM dan nama.
- `cursor.execute(query, (nim, nama))`: Mengeksekusi kueri dengan parameter NIM dan nama.
- `rows = cursor.fetchall()`: Menyimpan hasil kueri dalam variabel rows.

126-128. Memeriksa hasil kueri dan memberikan notifikasi

- `if len(rows) == 0`: Jika tidak ada data yang ditemukan (NIM atau nama tidak sesuai), maka:
- `notif1.set('Data tidak sesuai atau tidak ada')`: Mengatur teks notifikasi.
- `play_sound(success=False)`: Memutar suara kesalahan.

129-139. Proses presensi jika data valid

- `image = vid.read()`: Membaca frame dari kamera dan menyimpannya dalam variabel image.
- `current_datetime = datetime.now()`: Mendapatkan waktu saat ini.
- `formatted_datetime, current_time, current_date, current_day`: Memformat waktu saat ini menjadi berbagai format yang diperlukan.
- `cropped_image = image[y1:y2, x1:x2]`: Memotong bagian tertentu dari gambar (misalnya wajah) dan menyimpannya dalam `cropped_image`.
- `cv2.imwrite(f'file_{formatted_datetime}.png', cropped_image)`: Menyimpan gambar yang telah dipotong dengan nama file yang unik berdasarkan waktu.
- `sql = "INSERT INTO tb_presensi (id_user, hari_kedatangan, tanggal_kedatangan, waktu_kedatangan, image) VALUES (%s, %s, %s, %s, %s)"`: Mendefinisikan kueri SQL untuk memasukkan data presensi ke dalam database.
- `val = (nim, current_day, current_date, current_time, name_file)`: Menyiapkan nilai-nilai yang akan dimasukkan ke dalam kueri.
- `cursor.execute(sql, val)`: Mengeksekusi kueri untuk menyimpan data presensi.
- `cnx.commit()`: Melakukan commit perubahan ke database.

140-145. Menampilkan data presensi jika data valid

- `for row in rows`: Melakukan iterasi melalui setiap baris hasil kueri.
- `notif1.set('Nama: ' + row[1])`: Mengatur teks notifikasi 1 dengan nama mahasiswa.
- `notif2.set('NIM: ' + row[0])`: Mengatur teks notifikasi 2 dengan NIM mahasiswa.
- `notif3.set('Waktu: ' + current_time)`: Mengatur teks notifikasi 3 dengan waktu saat ini.
- `notif4.set('Hari: ' + current_day)`: Mengatur teks notifikasi 4 dengan hari ini.
- `notif5.set('Tanggal: ' + current_date)`: Mengatur teks notifikasi 5 dengan tanggal hari ini.

146-151. Menampilkan gambar dan notifikasi keberhasilan

- `img = Image.open(f'file_{formatted_datetime}.png')`: Membuka gambar yang telah disimpan sebelumnya.
- `resized_image = img.resize((125, 160))`: Mengubah ukuran gambar.
- `image_ = ImageTk.PhotoImage(resized_image)`: Mengubah gambar menjadi format yang dapat ditampilkan di GUI.
- `canvas1.create_image(40, 80, image=image_)`: Menampilkan gambar pada canvas.
- `notif6.set('Anda Berhasil Presensi')`: Mengatur teks notifikasi 6 dengan pesan keberhasilan.
- `play_sound(success=True)`: Memutar suara keberhasilan.

152. `cnx.close()`:

- Menutup koneksi ke database.

153-156. Menangani kasus koneksi gagal

- `else::` Jika koneksi ke database gagal.
- `notif1.set('Koneksi ke database gagal')`: Mengatur teks notifikasi dengan pesan kesalahan.
- `play_sound(success=False)`: Memutar suara kesalahan.

157-158. Menangani kasus NIM tidak valid

- `else::` Jika NIM tidak valid (bukan angka), maka:
- `notif1.set('Masukkan NIM dengan benar')`: Mengatur teks notifikasi dengan pesan kesalahan.
- `play_sound(success=False)`: Memutar suara kesalahan.

159. `def fill_nama_from_database()`:

- Mendefinisikan fungsi `fill_nama_from_database` untuk mengisi daftar nama mahasiswa dari database ke dalam komponen GUI (misalnya, ComboBox).

160-165. Mendefinisikan konfigurasi database

- Blok kode ini mendefinisikan detail konfigurasi untuk koneksi ke database MySQL.

166. `cnx = mysql.connector.connect(**config)`

- Mencoba membuat koneksi ke database MySQL menggunakan informasi konfigurasi yang telah didefinisikan sebelumnya.

167-170. Memeriksa koneksi dan mengeksekusi kueri

- `if cnx.is_connected()`: Memeriksa apakah koneksi ke database berhasil.
- `cursor = cnx.cursor()`: Membuat objek cursor untuk mengeksekusi kueri SQL.

- `query = "SELECT nama FROM tb_member"`: Mendefinisikan kueri SQL untuk mengambil semua nama mahasiswa.
- `cursor.execute(query)`: Mengeksekusi kueri.
- `rows = cursor.fetchall()`: Menyimpan hasil kueri dalam variabel `rows`.

171-173. Mengisi daftar nama ke komponen GUI

- `nama_list = [row[0] for row in rows]`: Membuat list berisi semua nama mahasiswa dari hasil kueri.
- `nama_menu['values'] = nama_list`: Mengisi nilai-nilai dalam ComboBox `nama_menu` dengan daftar nama yang telah dibuat.

174. `cnx.close()`

- Menutup koneksi ke database setelah proses presensi selesai atau setelah terjadi kesalahan koneksi.

175-179. Menangani kesalahan koneksi database

- `except mysql.connector.Error as err:` Blok `except` ini akan dijalankan jika terjadi kesalahan saat koneksi atau eksekusi kueri pada fungsi sebelumnya.
- `print(f'Error: {err}')`: Mencetak pesan kesalahan ke konsol untuk membantu debugging.

180. `def open_camera()`:

- Mendefinisikan fungsi `open_camera()` untuk memperbarui tampilan kamera secara terus-menerus.

181. `ret, frame = vid.read()`

- Membaca frame video dari kamera dan menyimpannya dalam variabel `frame`. Variabel `ret` menunjukkan apakah pembacaan frame berhasil.

182. `opencv_image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGBA)`

- Mengubah format warna frame dari BGR (OpenCV) menjadi RGBA (cocok untuk Tkinter) dan menyimpannya dalam `opencv_image`.

183. `captured_image = Image.fromarray(opencv_image)`

- Mengubah `opencv_image` menjadi format image library Pillow (PIL) dan menyimpannya dalam `captured_image`.

184. `photo_image = ImageTk.PhotoImage(image=captured_image)`

- Mengubah `captured_image` menjadi format image yang dapat ditampilkan di Tkinter dan menyimpannya dalam `photo_image`.

185. `canvas.create_image(100, 0, image=photo_image, anchor=tkinter.CENTER)`

- Menampilkan gambar `photo_image` pada canvas dengan posisi (100, 0) dan anchor (titik tumpu) di tengah.

186. `canvas.image = photo_image`

- menyimpan referensi ke image terbaru untuk keperluan update selanjutnya.

187. `tk.after(10, open_camera)`

- Menjadwalkan fungsi `open_camera` untuk dieksekusi kembali dalam 10 milidetik (10ms). Ini menciptakan efek pembaruan tampilan kamera secara terus-menerus.

188-193. Deklarasi `StringVar` untuk notifikasi

- Blok kode ini mendeklarasikan beberapa variabel `StringVar` untuk menyimpan teks notifikasi yang akan ditampilkan di label.

194-199. Meminisiasi nilai awal notifikasi

- Blok kode ini menginisiasi nilai awal untuk semua variabel `StringVar` dengan string kosong.

200-201. Mendefinisikan Canvas untuk menampilkan video feed

- Mendefinisikan sebuah canvas `canvas` dengan ukuran 360x350 pixel dan menempatkannya di koordinat tertentu menggunakan `pack` dan `place`.

202-203. Mendefinisikan Canvas untuk menampilkan gambar hasil tangkapan

- Mendefinisikan sebuah canvas `canvas1` dengan ukuran 100x120 pixel dan menempatkannya di koordinat tertentu menggunakan `pack` dan `place`.

204. Mendefinisikan Button untuk presensi

- Mendefinisikan tombol dengan teks "Presensi", ukuran 30x2 karakter, dan menghubungkannya dengan fungsi `process` ketika diklik. Tombol ini ditempatkan di koordinat tertentu menggunakan `place`.

205-208. Mendefinisikan Entry untuk input NIM

- Mendefinisikan widget Entry untuk pengguna memasukkan NIM. Lebarinya 29 karakter dan ditempatkan di koordinat tertentu menggunakan `place`.
- 209-212. Mendefinisikan Entry untuk input nama.
- Mendefinisikan widget Entry untuk pengguna memasukkan nama. Lebarinya 29 karakter dan ditempatkan di koordinat tertentu menggunakan `place`.
- 213-214. Mendefinisikan Entry untuk input prodi.
- Mendefinisikan widget Entry untuk pengguna memasukkan prodi. Lebarinya 29 karakter dan ditempatkan di koordinat tertentu menggunakan `place`.

215-218. Mendefinisikan ComboBox untuk Prodi

- `label_input_prodi`: Menciptakan label dengan teks "Pilih Prodi" untuk memberikan petunjuk kepada pengguna. Label ini ditempatkan di posisi (1270, 580) pada jendela aplikasi.

- `prodi_combobox`: Menciptakan sebuah ComboBox (dropdown list) yang berisi pilihan program studi (`prodi`). Nilai-nilai default yang diberikan adalah "Informatika", "Sistem Informasi", dan "Teknik Elektro". ComboBox ini ditempatkan di posisi (1200, 610) pada jendela aplikasi.

219-227. Mendefinisikan Label untuk Notifikasi

- `label_notif1`, `label_notif2`, dst.: Mencetak label-label untuk menampilkan berbagai jenis notifikasi. Variabel `textvariable` pada setiap label terhubung dengan `StringVar` yang telah didefinisikan sebelumnya (`notif1`, `notif2`, dst.).

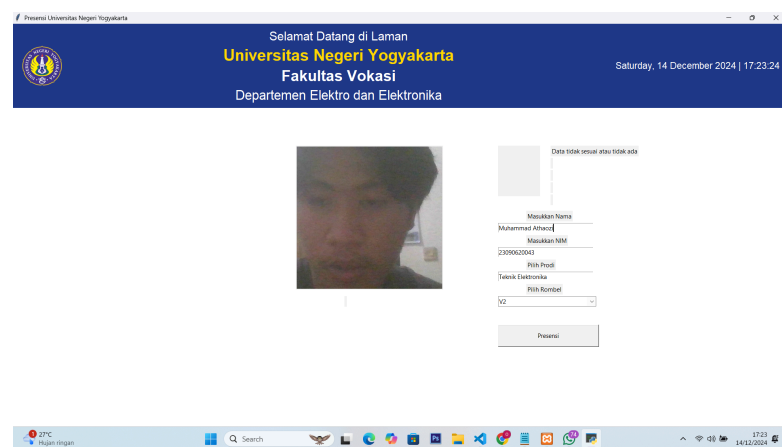
228-230. `tk.mainloop()`

- memulai event loop Tkinter. Event loop ini akan terus berjalan sampai aplikasi ditutup secara manual (misalnya dengan menekan tombol tutup pada jendela).
- Selama event loop berjalan, Tkinter akan terus memantau event-event yang terjadi (misalnya, klik tombol, perubahan input, dll.) dan menjalankan fungsi yang sesuai.

C. Hasil Program



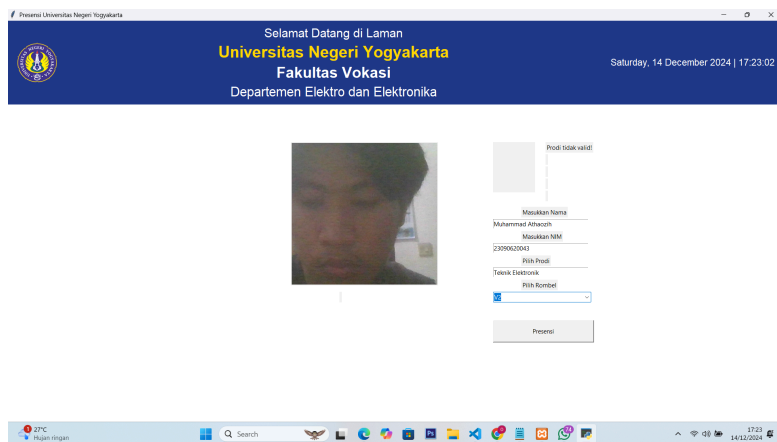
Gambar 2. Tampilan Awal Presensi



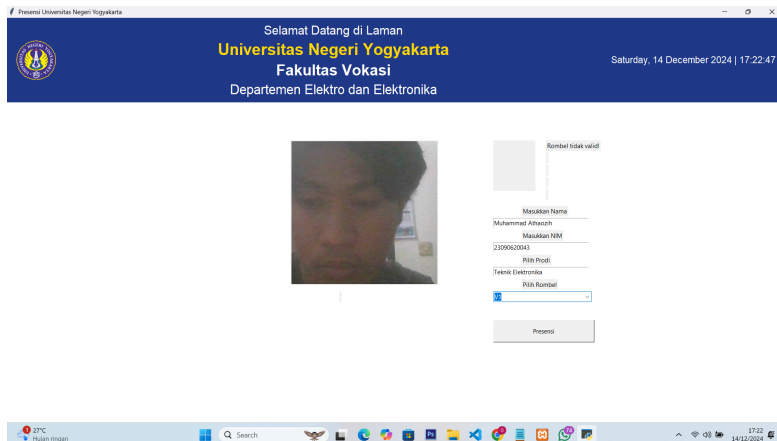
Gambar 3. Tampilan Presensi dengan Nama yang Salah



Gambar 4. Tampilan Presensi dengan NIM yang Salah



Gambar 5. Tampilan Presensi dengan Program Studi yang Salah

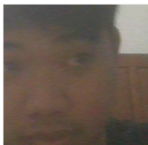


Gambar 6. Tampilan Presensi dengan Rombel yang Salah



Selamat Datang di Laman
Universitas Negeri Yogyakarta
Fakultas Vokasi
 Departemen Elektro dan Elektronika

Saturday, 14 December 2024 | 17:44:28



Anda Berhasil Presensi

Nama: Ahmad Iman Nadzion
 NIM: 2209020006
 Waktu: 17:44:24
 Hari: Saturday
 Tanggal: 2024-12-14

Masukkan Nama

Ahmad Iman Nadzion

Masukkan NIM

2209020006

Pilih Prodi

Teknik Elektronika

Pilih Rombel

V2

Presensi

The screenshot shows the phpMyAdmin interface with the 'information_schema' database selected. The 'tables' tab is active, displaying a list of tables. The table 'information_schema.TABLES' is highlighted. The table structure shows columns: TABLE_NAME, TABLE_SCHEMA, TABLE_TYPE, ENGINE, TABLE_COMMENT, and TABLE_ROWS. The table has 1 row and 6 columns.

Gambar 9. Tampilan Awal Database PHPMYADMIN

Showing rows 0 - 7 (8 total. Query took 0.0003 seconds.)

```
SELECT * FROM `tb_member`
```

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

id_nim	nama_mahasiswa	Prodi	Rombel	Tanggal_Lahir
23090620043	Muhammad Athazoh	Teknik Elektronika V2		2015-10-07 16:17:38
23090620043	Muhammad Athazoh	Teknik Elektronika V2		2015-10-07 16:17:38
23090620065	Ahmad Ilman Nadziron	Teknik Elektronika V2		2015-12-10 16:18:40
23090620065	Ahmad Ilman Nadziron	Teknik Elektronika V2		2015-12-10 16:18:40
23090620078	Zacky Alamsyah	Teknik Elektronika V2		2019-05-15 16:21:02
23090620078	Zacky Alamsyah	Teknik Elektronika V2		2019-05-15 16:21:02
23090620084	Anjani Nuria Arsyia	Teknik Elektronika V2		2018-12-17 16:22:14
23090620084	Anjani Nuria Arsyia	Teknik Elektronika V2		2018-12-17 16:22:14

Gambar 10. Tampilan Database “tb_member”

id_user	hari_kedatangan	tanggal_kedatangan	waktu_kedatangan	image
23090620043	Tuesday	2024-11-26	06:48:19	file_20241126064819.png
23090620043	Tuesday	2024-11-26	06:50:00	file_20241126065000.png
23090620065	Tuesday	2024-11-26	06:50:09	file_20241126065009.png
23090620043	Tuesday	2024-11-26	06:53:25	file_20241126065325.png
23090620043	Tuesday	2024-11-26	06:56:34	file_20241126065634.png
23090620043	Tuesday	2024-11-26	06:57:29	file_20241126065729.png
23090620043	Tuesday	2024-11-26	06:58:30	file_20241126065830.png
23090620043	Tuesday	2024-11-26	06:59:09	file_20241126065909.png
23090620043	Tuesday	2024-11-26	06:59:46	file_20241126065946.png
23090620065	Tuesday	2024-11-26	08:08:42	file_20241126080842.png
23090620065	Tuesday	2024-11-26	08:32:46	file_20241126083246.png
23090620065	Tuesday	2024-11-26	08:32:51	file_20241126083251.png
23090620065	Tuesday	2024-11-26	08:33:19	file_20241126083319.png
23090620078	Tuesday	2024-11-26	08:34:58	file_20241126083458.png
23090620078	Tuesday	2024-11-26	08:34:58	file_20241126083458.png
23090620043	Tuesday	2024-11-26	08:48:38	file_20241126084838.png
23090620043	Tuesday	2024-11-26	08:52:14	file_20241126085214.png
23090620043	Tuesday	2024-11-26	08:57:40	file_20241126085740.png
23090620065	Tuesday	2024-11-26	09:05:16	file_20241126090516.png
23090620043	Saturday	2024-12-14	14:30:27	file_20241214143027.png
23090620043	Saturday	2024-12-14	14:30:56	file_20241214143056.png
23090620043	Saturday	2024-12-14	14:38:29	file_20241214143829.png
23090620043	Saturday	2024-12-14	14:51:23	file_20241214145123.png
23090620043	Saturday	2024-12-14	15:15:54	file_20241214151554.png
23090620065	Saturday	2024-12-14	15:21:46	file_20241214152146.png
23090620065	Saturday	2024-12-14	15:21:58	file_20241214152158.png
23090620043	Saturday	2024-12-14	15:54:06	file_20241214155406.png
23090620043	Saturday	2024-12-14	16:02:19	file_20241214160219.png
23090620043	Saturday	2024-12-14	16:24:37	file_20241214162437.png
23090620043	Saturday	2024-12-14	16:25:02	file_20241214162502.png
23090620043	Saturday	2024-12-14	16:36:43	file_20241214163643.png
23090620065	Saturday	2024-12-14	16:37:09	file_20241214163709.png
23090620043	Saturday	2024-12-14	17:04:31	file_20241214170431.png
23090620043	Saturday	2024-12-14	17:04:40	file_20241214170440.png
23090620043	Saturday	2024-12-14	17:04:44	file_20241214170444.png
23090620043	Saturday	2024-12-14	17:04:47	file_20241214170447.png
23090620043	Saturday	2024-12-14	17:04:52	file_20241214170452.png
23090620043	Saturday	2024-12-14	17:15:27	file_20241214171527.png
23090620043	Saturday	2024-12-14	17:23:33	file_20241214172333.png
23090620078	Saturday	2024-12-14	17:34:23	file_20241214173423.png
23090620078	Saturday	2024-12-14	17:34:34	file_20241214173434.png
23090620065	Saturday	2024-12-14	17:44:14	file_20241214174414.png
23090620065	Saturday	2024-12-14	17:44:24	file_20241214174424.png

Gambar 11. Tampilan Database “tb_presensi”

D. Pembahasan



Gambar 12. Tampilan Presensi

- Saat program dijalankan, **Tkinter** memulai proses pembuatan GUI (Graphical User Interface).
- GUI dibagi menjadi beberapa komponen:
 1. **Header:** Menampilkan logo institusi, judul program, dan waktu real-time.
 2. **Form Input:** Kolom input untuk **NIM**, **Nama**, dan **Prodi**. Dropdown disediakan untuk memilih **Rombel**.
 3. **Kamera Feed:** Area di mana tampilan real-time dari kamera ditampilkan menggunakan **OpenCV**.
 4. **Output Informasi:** Area untuk menampilkan hasil presensi, seperti notifikasi status, data yang telah divalidasi, dan foto yang berhasil diambil.
- Fungsi `update_time()` dijalankan dalam loop untuk menampilkan **waktu real-time** yang terus diperbarui setiap detik.
- **User** diminta untuk memasukkan data yang diperlukan:
 1. **NIM** (Nomor Induk Mahasiswa)
 2. **Nama**
 3. **Program Studi (Prodi)**
 4. **Rombel** (dipilih melalui dropdown)
- Data ini dikumpulkan melalui input field **Entry Widget** yang disediakan Tkinter.
- Setelah pengguna memasukkan semua data, program akan menunggu tombol **Presensi** ditekan. Tombol ini memicu fungsi utama `process()`.
- Setelah tombol **Presensi** ditekan:
- Fungsi `validate_prodi()` dan `validate_rombel()` memeriksa apakah **Program Studi** dan **Rombel** yang dimasukkan ada di dalam database.

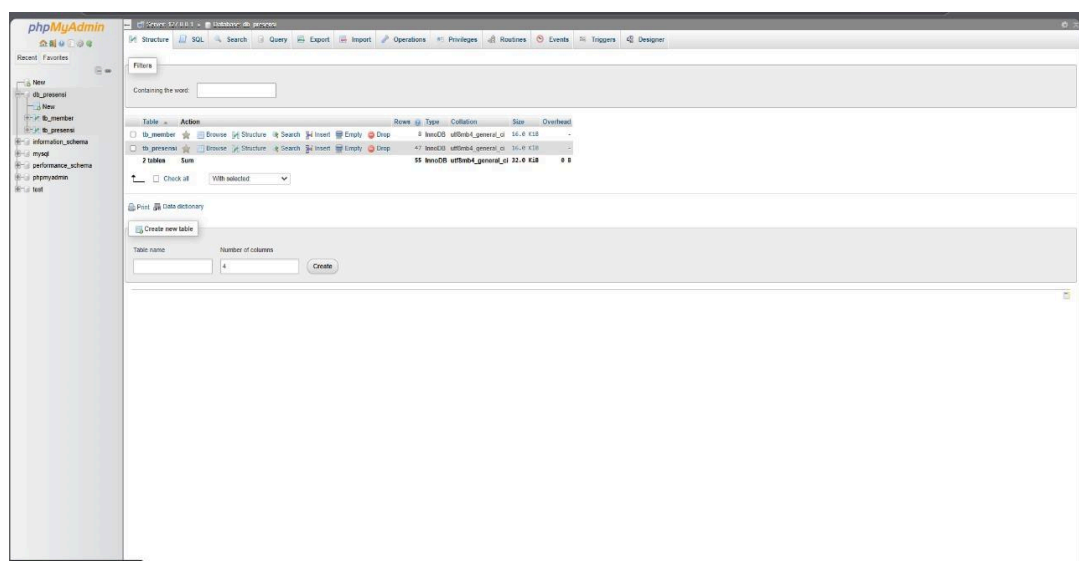
- Jika **Prodi** atau **Rombel** tidak valid:
 - Program akan menghentikan proses.
 - Pesan notifikasi error ditampilkan pada GUI.
 - Suara "gagal" akan diputar menggunakan pustaka **Pygame**.
- Jika valid, program lanjut ke langkah berikutnya.
- Data **NIM** dan **Nama** diperiksa di tabel **tb_member** yang sudah disiapkan di MySQL.
- Query SQL dijalankan untuk mencari data

```
SELECT * FROM tb_member WHERE nim = 'input_NIM' AND nama = 'input_Nama';
```

- Jika data tidak ditemukan:
 - Program menghentikan proses.
 - Menampilkan notifikasi "Data tidak sesuai atau tidak ada."
 - Memutar suara "gagal" menggunakan **Pygame**.
- Jika data valid, program melanjutkan ke langkah berikutnya.
- Jika data dinyatakan valid, program mengaktifkan kamera yang terhubung melalui **OpenCV**.
- Kamera akan menampilkan feed real-time di area yang disediakan pada GUI.
- Gambar akan diambil dan disimpan ketika semua validasi berhasil.
- Gambar ditangkap menggunakan fungsi `cv2.VideoCapture()` dari OpenCV.
- Gambar disimpan dengan nama unik berdasarkan waktu saat presensi dilakukan. Contoh:

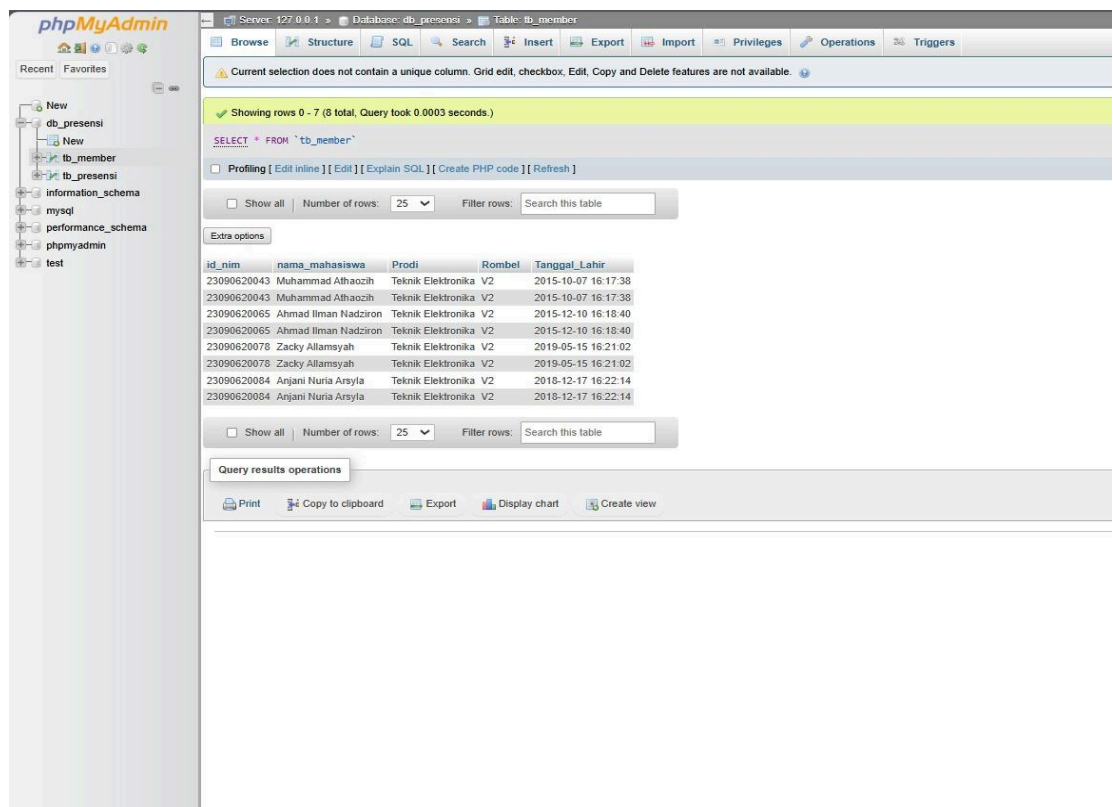
```
filename = f"presensi_{waktu}.png"
cv2.imwrite(filename, frame)
```

- File gambar akan disimpan di folder kerja program



Gambar 13. Database

- Dalam sistem database yang ada di gambar 13, terdapat **dua tabel utama** yang memiliki hubungan atau relasi, yaitu **tb_member** dan **tb_presensi**. Tabel-tabel ini bekerja bersama-sama untuk mengelola data keanggotaan dan mencatat kehadiran (presensi) mahasiswa.
- Database adalah sebuah sistem terstruktur yang digunakan untuk menyimpan, mengatur, dan mengelola data agar bisa diakses dengan mudah, cepat, dan aman. Dalam konteks ini, database berfungsi untuk:
- Menyimpan **data anggota** (dalam tabel tb_member).
- Menyimpan **data kehadiran** yang dicatat setiap kali anggota melakukan presensi (dalam tabel tb_presensi).
- Menghubungkan kedua tabel tersebut agar dapat berinteraksi satu sama lain dengan baik.



The screenshot shows the phpMyAdmin interface for a database named 'db_presensi'. The 'tb_member' table is selected, and its structure and data are displayed. The table has five columns: id_nim, nama_mahasiswa, Prodi, Rombel, and Tanggal_Lahir. The data shows 8 rows of student information.

id_nim	nama_mahasiswa	Prodi	Rombel	Tanggal_Lahir
23090620043	Muhammad Athaozh	Teknik Elektronika	V2	2015-10-07 16:17:38
23090620043	Muhammad Athaozh	Teknik Elektronika	V2	2015-10-07 16:17:38
23090620065	Ahmad Ilman Nadziron	Teknik Elektronika	V2	2015-12-10 16:18:40
23090620065	Ahmad Ilman Nadziron	Teknik Elektronika	V2	2015-12-10 16:18:40
23090620078	Zacky Allamsyah	Teknik Elektronika	V2	2019-05-15 16:21:02
23090620078	Zacky Allamsyah	Teknik Elektronika	V2	2019-05-15 16:21:02
23090620084	Anjani Nuria Arysya	Teknik Elektronika	V2	2018-12-17 16:22:14
23090620084	Anjani Nuria Arysya	Teknik Elektronika	V2	2018-12-17 16:22:14

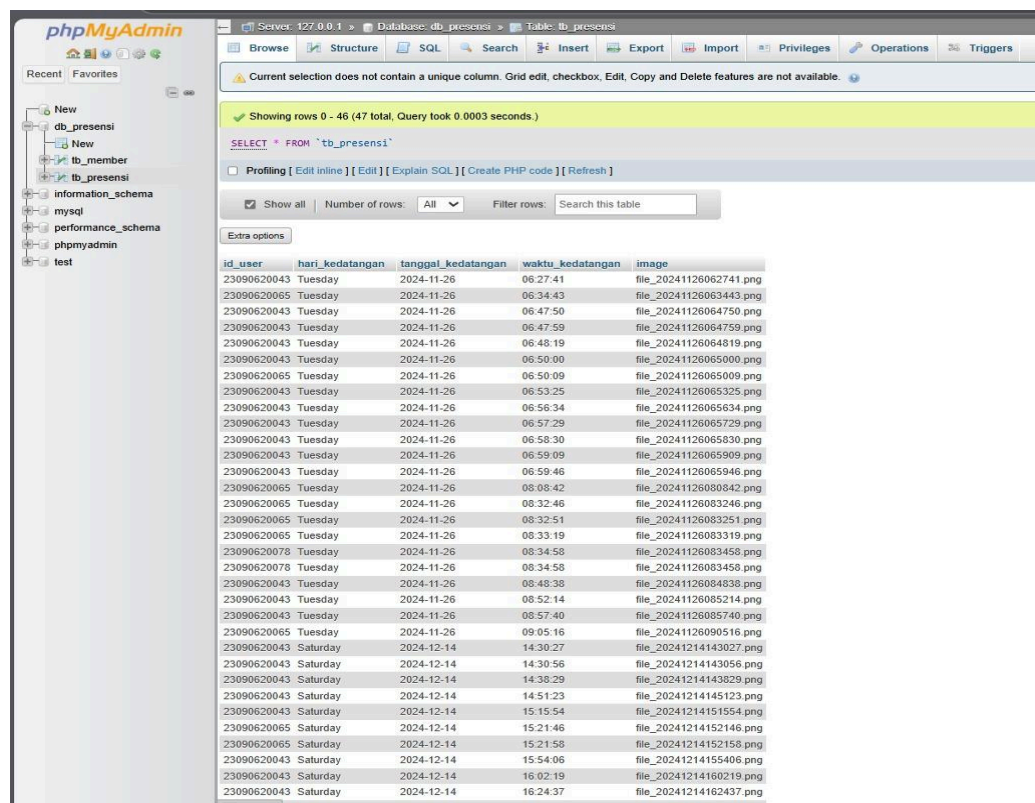
Gambar 14. tb_member

- Tabel tb_member berperan sebagai **data master** atau induk yang menyimpan informasi lengkap tentang anggota (misalnya mahasiswa atau peserta). Tabel ini berfungsi sebagai referensi untuk memvalidasi identitas seseorang sebelum mereka dapat melakukan presensi.
- Program menyimpan data presensi ke tabel tb_presensi di database MySQL. Data yang disimpan meliputi:
 - **NIM**

- Nama
- Prodi
- Rombel
- Tanggal dan Waktu Presensi
- Lokasi File Gambar (Path gambar yang diambil)
- Query SQL digunakan untuk menyimpan data:

```
INSERT INTO tb_member (id_nim, nama, prodi, rombel, status)
VALUES ('23090620084', 'Anjani Nuria Arsyla', 'Teknik Elektronika', 'V2',
'Aktif');
```

- Jika penyimpanan berhasil:
 - Program menampilkan notifikasi "**Presensi Berhasil**".
 - Data presensi ditampilkan kembali di GUI, termasuk gambar yang diambil.
- Jika semua langkah berjalan dengan baik
 - Pesan **Presensi Berhasil** ditampilkan.
 - Gambar pengguna akan ditampilkan di GUI.
 - Suara "berhasil" diputar menggunakan **Pygame**.
- Jika ada kegagalan dalam input atau validasi
 - Pesan error akan ditampilkan.
 - Suara "gagal" diputar sebagai notifikasi.



The screenshot shows the phpMyAdmin interface with the 'tb_presensi' table selected. The table contains 46 rows of data. The columns are: id_user, hari_kedatangan, tanggal_kedatangan, waktu_kedatangan, and image. The data shows attendance records for various users on Tuesdays and Saturdays in 2024.

id_user	hari_kedatangan	tanggal_kedatangan	waktu_kedatangan	image
23090620043	Tuesday	2024-11-26	06:27:41	file_20241126062741.png
23090620065	Tuesday	2024-11-26	06:34:43	file_20241126063443.png
23090620043	Tuesday	2024-11-26	06:47:50	file_20241126064750.png
23090620043	Tuesday	2024-11-26	06:47:59	file_20241126064759.png
23090620043	Tuesday	2024-11-26	06:48:19	file_20241126064819.png
23090620043	Tuesday	2024-11-26	06:50:00	file_20241126065000.png
23090620065	Tuesday	2024-11-26	06:50:09	file_20241126065009.png
23090620043	Tuesday	2024-11-26	06:53:25	file_20241126065325.png
23090620043	Tuesday	2024-11-26	06:56:34	file_20241126065634.png
23090620043	Tuesday	2024-11-26	06:57:29	file_20241126065729.png
23090620043	Tuesday	2024-11-26	06:58:30	file_20241126065830.png
23090620043	Tuesday	2024-11-26	06:59:09	file_20241126065909.png
23090620043	Tuesday	2024-11-26	06:59:46	file_20241126065946.png
23090620065	Tuesday	2024-11-26	08:08:42	file_20241126080842.png
23090620065	Tuesday	2024-11-26	08:32:46	file_20241126083246.png
23090620065	Tuesday	2024-11-26	08:32:51	file_20241126083251.png
23090620065	Tuesday	2024-11-26	08:33:19	file_20241126083319.png
23090620078	Tuesday	2024-11-26	08:34:58	file_20241126083458.png
23090620078	Tuesday	2024-11-26	08:34:58	file_20241126083458.png
23090620043	Tuesday	2024-11-26	08:48:38	file_20241126084838.png
23090620043	Tuesday	2024-11-26	08:52:14	file_20241126085214.png
23090620043	Tuesday	2024-11-26	08:57:40	file_20241126085740.png
23090620065	Tuesday	2024-11-26	09:05:16	file_20241126090516.png
23090620043	Saturday	2024-12-14	14:30:27	file_20241214143027.png
23090620043	Saturday	2024-12-14	14:30:56	file_20241214143056.png
23090620043	Saturday	2024-12-14	14:38:29	file_20241214143829.png
23090620043	Saturday	2024-12-14	14:51:23	file_20241214145123.png
23090620043	Saturday	2024-12-14	15:15:54	file_20241214151554.png
23090620065	Saturday	2024-12-14	15:21:46	file_20241214152146.png
23090620065	Saturday	2024-12-14	15:21:58	file_20241214152158.png
23090620043	Saturday	2024-12-14	15:54:06	file_20241214155406.png
23090620043	Saturday	2024-12-14	16:02:19	file_20241214160219.png
23090620043	Saturday	2024-12-14	16:24:37	file_20241214162437.png

Gambar 3. *tb_presensi*

- Tabel `tb_presensi` berfungsi untuk **mencatat data kehadiran** atau presensi dari anggota yang valid. Tabel ini merekam informasi tentang siapa yang hadir, kapan hadir, dan foto bukti kehadiran.
- Data yang tersimpan meliputi:
 - **id_presensi** (*Primary Key*): ID unik untuk setiap catatan presensi. Ini akan membedakan satu presensi dengan presensi lainnya.
 - **id_user** (*Foreign Key*): Merujuk ke kolom `id_nim` di tabel `tb_member`. Ini memastikan bahwa presensi hanya bisa dilakukan oleh anggota yang ada di tabel `tb_member`.
 - **hari_kedatangan**: Hari saat presensi dilakukan (misalnya: Senin, Selasa, dsb).
 - **tanggal_kedatangan**: Tanggal kehadiran (format: YYYY-MM-DD).
 - **waktu_kedatangan**: Waktu saat presensi dilakukan (format: HH:MM:SS).
 - **image**: Nama file foto yang diambil sebagai bukti presensi.
- Saat anggota melakukan presensi, sistem akan memeriksa apakah **NIM** yang dimasukkan ada di tabel `tb_member`.
- Jika valid, maka proses dilanjutkan.
- Sistem akan merekam:
 - **Hari**: Diambil dari sistem waktu komputer atau server.
 - **Tanggal**: Tanggal saat presensi dilakukan.
 - **Waktu**: Waktu (jam, menit, detik) saat presensi terjadi.
- Foto anggota akan diambil melalui kamera (misalnya menggunakan perangkat lunak seperti OpenCV), lalu disimpan dengan nama unik, seperti:

```
image_20240618_093015.png
```

- Data yang sudah terkumpul akan dimasukkan ke tabel `tb_presensi` dengan perintah SQL

```
INSERT INTO tb_presensi (id_user, hari_kedatangan, tanggal_kedatangan, waktu_kedatangan, image)
VALUES ('12345678', 'Senin', '2024-06-18', '09:30:15', 'image_20240618_093015.png');
```

- Kolom **id_user** di `tb_presensi` memiliki hubungan dengan **id_nim** di `tb_member`. Hubungan ini memastikan bahwa:
 - Data presensi selalu dikaitkan dengan anggota yang ada di `tb_member`.
 - Presensi tidak dapat dilakukan oleh anggota yang tidak valid.
- Administrator bisa menampilkan data presensi menggunakan query seperti ini:

```
SELECT tb_presensi.*, tb_member.nama, tb_member.prodi,
tb_member.rombel
```

```
FROM tb_presensi  
JOIN tb_member ON tb_presensi.id_user = tb_member.id_nim;
```

- Hasilnya akan menampilkan data lengkap kehadiran, termasuk nama anggota, hari, tanggal, waktu, dan foto bukti.

Dari percobaan yang telah dilakukan, ketika Nama, NIM, Program Studi, dan Rombel dimasukkan ke dalam tampilan presensi dengan karakteristik yang berbeda dengan database di “tb_member”, maka presensinya tidak berhasil. Ketika Nama, dan NIM salah, maka di tampilan presensi akan menampilkan “Data tidak sesuai atau tidak ada”. Jika Program Studi yang dimasukkan salah, maka akan menampilkan “Prodi tidak valid!”. Jika Rombel yang dimasukkan salah, maka akan menampilkan “Rombel tidak valid!”. Ketika Nama, NIM, Prodi, dan Rombel dimasukkan sesuai dengan yang ada di database “tb_member”, maka presensi akan berhasil dengan menampilkan “Anda Berhasil Presensi”. Presensi yang telah berhasil akan dimasukkan ke database “tb_presensi” yang digunakan untuk menyimpan data yang telah presensi.

E. Kesimpulan

Sistem presensi mahasiswa dengan berbasis database dan GUI merupakan sebuah aplikasi berbasis komputer yang dibangun dengan tujuan untuk memudahkan proses pencatatan kehadiran mahasiswa di Departemen Elektro dan Elektronika, Fakultas Vokasi, Universitas Negeri Yogyakarta. Aplikasi ini dirancang untuk memungkinkan mahasiswa melakukan presensi secara digital dengan cara memasukkan informasi pribadi seperti NIM (Nomor Induk Mahasiswa), nama, program studi yang ditempuh, serta kelas Rombel Praktek. Setelah mahasiswa memasukkan data tersebut, sistem akan secara otomatis mencatat waktu kehadiran, hari, serta tanggal saat presensi dilakukan, dan juga menyimpan foto dari proses presensi ke dalam basis data yang terintegrasi.

Aplikasi ini tidak hanya berfungsi sebagai alat pencatat kehadiran, tetapi juga dilengkapi dengan beberapa fitur tambahan yang meningkatkan efisiensi dan akurasi keseluruhan sistem. Misalnya, terdapat penampil waktu dan tanggal terkini yang memungkinkan pengguna untuk selalu mengetahui informasi terkini. Selain itu, sistem juga dilengkapi dengan notifikasi yang akan memberikan umpan balik kepada pengguna, baik itu terkait keberhasilan maupun kegagalan proses presensi yang telah dilakukan. Dengan demikian, aplikasi ini diharapkan dapat membantu memastikan bahwa proses pencatatan kehadiran mahasiswa berjalan dengan lancar dan akurat.

Penggunaan teknologi digital dan basis data dalam membangun aplikasi ini menunjukkan adanya upaya untuk mengelola informasi presensi secara terstruktur dan terorganisir. Hal ini tentunya akan memberikan manfaat bagi pihak-pihak yang terlibat, baik itu mahasiswa, staf administrasi, maupun pihak manajemen Departemen Elektro dan Elektronika, Fakultas Vokasi, Universitas Negeri Yogyakarta, dalam memantau dan mengelola data kehadiran mahasiswa dengan lebih efektif dan efisien.

F. Referensi

- Triwibowo, Arif. *RANCANG BANGUN SISTEM PRESENSI DIGITAL MAHASISWA BERBASIS WEB DI GEDUNG ELEKTRONIKA*. Diss. POLITEKNIK NEGERI SRIWIJAYA, 2020.
- Liana, Dhewi April, et al. "Sistem Presensi Mahasiswa Menggunakan Fitur Deteksi Wajah Berbasis Cognitive Internet of Things." *Jurnal Pekommas* 8.2 (2023): 129-136.
- Wiryadinata, Romi, et al. "Sistem Presensi Menggunakan Algoritme Eigenface dengan Deteksi Aksesoris dan Ekspresi Wajah." *Jurnal Nasional Teknik Elektro dan Teknologi Informasi* (2017).